

Grant Harvey
harveyg3@udayton.edu
CPS 475 - Secure App Development
Dr Phu Phung

Lab 02

Task 01 - Exploiting and fixing vulnerable C programs

a: Format String Vulnerability

i:

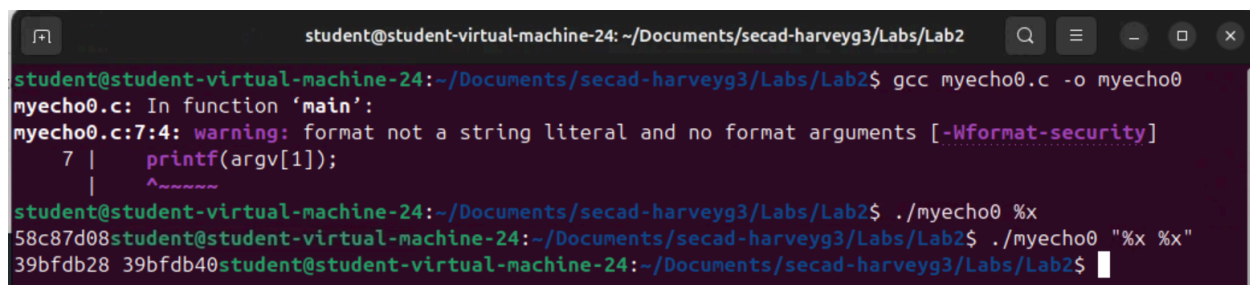
Link to commit with vulnerability

<https://bitbucket.org/granthsrvey/secad-harveyg3/commits/f947e6ccefc50231fbe72862239f4ab94adc8c30>



```
myecho0.c
1 /* include libraries */
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main (int argc, char *argv[])
6 {
7     printf(argv[1]);
8 }
9
```

```
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/Lab2
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$ gcc myecho0.c
myecho0.c: In function 'main':
myecho0.c:7:4: warning: format not a string literal and no format arguments [-Wformat-security]
7 |     printf(argv[1]);
  |     ^
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$ ./a.out secad
secadstudent@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$
```



```
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/Lab2
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$ gcc myecho0.c -o myecho0
myecho0.c: In function 'main':
myecho0.c:7:4: warning: format not a string literal and no format arguments [-Wformat-security]
7 |     printf(argv[1]);
  |     ^
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$ ./myecho0 %x
58c87d08student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$ ./myecho0 "%x %x"
39bfdb28 39bfdb40student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$
```

The format string vulnerability occurs because `argv[1]` is passed directly as the format argument to `printf()` without a specified format string. When a user inputs format specifiers such as `%x` or `%s`, `printf()` interprets them as instructions to retrieve data from the stack. `printf()` then reads and prints memory values directly off the call stack, leading to information leakage.

ii:

Link to commit without vulnerability

<https://bitbucket.org/granthsrvey/secad-harveyg3/commits/55fd73891c4ced67d24b35a341f73bb3c934411b>

```
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/Lab2
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$ gcc myecho0.c -o myecho0
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$ ./myecho0 %x
%x
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$ ./myecho0 "%x %s"
%x %s
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$
```

Code Dif in bitbucket:

▼  Labs / Lab2 / myecho0.c 

↑	@@ -4,5 +4,5 @@ (Shift + click to expand all lines)
4 4	
5 5	int main (int argc, char *argv[])
6 6	{
7 -	printf(argv[1]);
7 +	printf("%s\n", argv[1]);
8 8	}
↓	(Shift + click to expand all lines)

The code was updated to `printf("%s\n", argv[1]);`. By supplying `%s` as a hardcoded format specifier string literal, `printf()` treats `argv[1]` purely as data (a string to print) rather than format control instructions. Any format specifiers like `%x` inside `argv[1]` are printed literally instead of being evaluated, effectively removing the vulnerability.

b: Buffer Overflow Vulnerability

i: Updated code as specified and now this is the output

```
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$ gcc -o myecho myecho.c
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$ ./myecho $(perl -e "print 'A'x137")
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAA
*** stack smashing detected ***: terminated
Aborted (core dumped)
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$
```

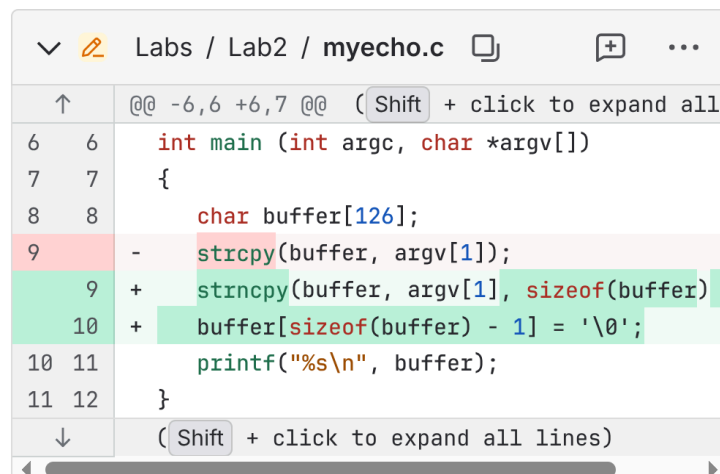
Commit link:

<https://bitbucket.org/granthsrvey/secad-harveyg3/commits/242c5a0883284e3246a37e66341fb2c488deca79>

The buffer overflow vulnerability occurs because strcpy() copies the command-line argument argv[1] into the fixed-size buffer array (126 bytes) without performing bounds checking. When an input larger than 126 bytes (such as 137 characters) is supplied, strcpy() writes past the allocated memory boundary of buffer on the stack.

ii: modify myecho.c to avoid buffer overflow vulnerability

```
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$ gcc -o myecho myecho.c
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$ ./myecho $(perl -e "print 'A'x137")
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAA
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$
```



The screenshot shows a code editor with the file 'Labs / Lab2 / myecho.c' open. The code is as follows:

```
6 6 int main (int argc, char *argv[])
7 7 {
8 8     char buffer[126];
9 9 - strcpy(buffer, argv[1]);
9 9 + strncpy(buffer, argv[1], sizeof(buffer))
10 10 + buffer[sizeof(buffer) - 1] = '\0';
10 11     printf("%s\n", buffer);
11 12 }
```

The editor interface includes line numbers, a diff view with '+' and '-' signs, and a status bar at the bottom indicating '(Shift + click to expand all lines)'.

Commit Link:

<https://bitbucket.org/granthsrvey/secad-harveyg3/commits/5ab2f730515e0d6acb022f00e0ccd3e5b3c8491a>

I fixed the vulnerability by replacing strcpy() with strncpy() with the max size of the buffer which prevents the buffer overflow vulnerability by forcing a strict limit on the number of bytes copied into buffer.

i: This function is not vulnerable to a format string vulnerability, we can tell because it is able to print out %x and %s

ii: This function is vulnerable to a buffer overflow vulnerability, we can tell because it is not able to print out A 1376 times

[illegible]

iii: Make a shell/bash script to find the max input size

```
student@student-virtual-machine-24:~/Documents/secad/labs/lab2$ ./bof-detect.sh
./myecho
Detecting max input size for './myecho' program by Grant Harvey
x
Executed successfully with input length = 1
xx
Executed successfully with input length = 2
xxx
Executed successfully with input length = 3
xxxx
Executed successfully with input length = 4
xxxxx
Executed successfully with input length = 5
xxxxxx

Executed successfully with input length = 488
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
*** stack smashing detected ***: terminated
./bof-detect.sh: line 25: 157397 Aborted                  (core dumped) $program
python3 -c "print('x'*$length)"
Failed. Max input length = 488
```

Source Code:

```
#!/bin/bash
program=$1
if [[ -z "$program" ]]; then
    echo "Usage: $0 <program>"
    exit 1
fi
echo "Detecting max input size for '$program' program by Grant Harvey"
length=1
while true
do
    $program "`python3 -c "print('x'*$length)""`
    return_code=$?
    if [ $return_code -eq 127 ]; then
        echo "Program does not exist! Try again."
        exit 2
    elif [ $return_code -eq 0 ]
    then
        printf "Executed successfully with input length = $length\n"
        length=$((length+1))
    else
        size=$((length-1))
        printf "Failed. Max input length = $size\n"
        break
    fi
done
```

Commit Link:

<https://bitbucket.org/granthsrvey/secad-harveyg3/commits/4f191903d3bc65f3f7f412feea1e18ce4b98d2d0>

Task 2 - A Simple TCP client application in C

a: Test your Program

```
student@student-virtual-machine-24:~/Documents/secad-harvey
g3/Labs/Lab2$ gcc mytcpclient.c -o mytcpclient
student@student-virtual-machine-24:~/Documents/secad-harvey
g3/Labs/Lab2$ ./mytcpclient localhost 8000
TCP Client Program by Grant Harvey
Server Name = localhost, Port Number = 8000
A socket is opened!
Connected to the server localhost at port 8000
Enter your message to send: Hello from Grant Harvey
Received from server: Hello from Grant Harvey
student@student-virtual-machine-24:~/Documents/secad-harvey
g3/Labs/Lab2$
```

```
student@student-virtual-machine-24:~/Documents/secad-harvey
g3/lectures/lecture3-review$ java EchoServer 8000
EchoServer is running at port 8000
A client is connected
received from client: Hello from Grant Harvey
Echo back
student@student-virtual-machine-24:~/Documents/secad-harvey
g3/lectures/lecture3-review$
```

b: Source Code

```
/* include libraries */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <unistd.h>
#include <netdb.h>

int main (int argc, char *argv[])
{
    /* Start Script and end code if user input is incorrect */
    if (strlen(argv[1], 254) > 253 || strlen(argv[2], 6) > 5)
    {
        printf("Servername or port is too long. Please try again!\n");
        exit(-1);
    }
    if (argc != 3)
    {
        printf("Usage: %s <servername> <port>\n", argv[0]);
        exit(1);
    }
    printf("TCP Client Program by Grant Harvey\n");
    printf("Server Name = %s, Port Number = %s\n", argv[1], argv[2]);

    /* Socket Stuff Below */
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("ERROR opening socket");
        exit(sockfd);
    }
    printf("A socket is opened!\n");

    /* Get address */
    struct addrinfo hints, *serveraddr;
```

```

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_INET;
hints.ai_socktype = SOCK_STREAM;
int addr_lookup = getaddrinfo(argv[1], argv[2], &hints, &serveraddr);
if (addr_lookup != 0)
{
    fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(addr_lookup));
    exit(addr_lookup);
}

/* Connect to Address */
int connected = connect(sockfd, serveraddr->ai_addr, serveraddr->ai_addrlen);
if (connected < 0)
{
    perror("Cannot connect to the server\n");
    exit(connected);
}
else
{
    printf("Connected to the server %s at port %s\n", argv[1], argv[2]);
}
freeaddrinfo(serveraddr);

/* Send data from user input */
int BUFFERSIZE = 1024;
char buffer[BUFFERSIZE];
bzero(buffer, BUFFERSIZE);
printf("Enter your message to send: ");
fgets(buffer, BUFFERSIZE, stdin);
int byte_sent = send(sockfd, buffer, strlen(buffer), 0);

/* Recieve Data */
bzero(buffer, BUFFERSIZE);
int byte_recieved = recv(sockfd, buffer, BUFFERSIZE, 0);
if(byte_recieved < 0)
{
    perror("Error in Reading");
    exit(byte_recieved);
}
printf("Recieved from server: %s", buffer);
close(sockfd);
return 0;

/* Close Socket */
close(sockfd);
printf("A socket is closed!\n");
}

```

Task 3 - A Simple yet secure HTTP client application in C.

a: Test your Program

```
student@student-virtual-machine-24:~/Documents/secad-harvey
g3/Labs/Lab2$ gcc myhttpclient.c -o myhttpclient
student@student-virtual-machine-24:~/Documents/secad-harvey
g3/Labs/Lab2$ ./myhttpclient example.com /index.html
HTTP Client Program by Grant Harvey
Server Name = example.com, Path = /index.html
A socket is opened!
Connected to the server example.com at port http-80
HTTP Request Sent:
GET /index.html HTTP/1.1
Host: example.com

HTTP Response Recieved:
HTTP/1.1 404 Not Found
Date: Wed, 16 Sep 2026 18:09:38 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: keep-alive
Server: cloudflare
Age: 8144
cf-cache-status: HIT
CF-RAY: a3c1d2c61b5e8e5c-ORD

22f
<!doctype html><html lang="en"><head><title>Example Domain<
/title><link rel="icon" href="data:,"><meta name="viewport"
content="width=device-width, initial-scale=1"><style>body{
background:#eee;width:60vw;margin:15vh auto;font-family:sys
```

Source Code:

```
/* include libraries */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <unistd.h>
#include <netdb.h>

int main (int argc, char *argv[])
{
    /* Start Script and end code if user input is incorrect */
    if (strlen(argv[1], 254) > 253 || strlen(argv[2], 254) > 253)
    {
        printf("Servername or path is too long. Please try again!\n");
        exit(-1);
    }
    if (argc != 3)
    {
        printf("Usage: %s <servername> <path>\n", argv[0]);
        exit(1);
    }
    printf("HTTP Client Program by Grant Harvey\n");
    printf("Server Name = %s, Path = %s\n", argv[1], argv[2]);

    /* Socket Stuff Below */
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("ERROR opening socket");
        exit(sockfd);
    }
```

```

}
printf("A socket is opened!\n");

/* Get address */
struct addrinfo hints, *serveraddr;
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_INET;
hints.ai_socktype = SOCK_STREAM;
int addr_lookup = getaddrinfo(argv[1], "80", &hints, &serveraddr);
if (addr_lookup != 0) {
    fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(addr_lookup));
    close(sockfd);
    exit(addr_lookup);
}

/* Connect to Address */
int connected = connect(sockfd, serveraddr->ai_addr, serveraddr->ai_addrlen);
if (connected < 0) {
    perror("Cannot connect to the server\n");
    freeaddrinfo(serveraddr);
    close(sockfd);
    exit(connected);
}
else {
    printf("Connected to the server %s at port http-80\n", argv[1]);
}
freeaddrinfo(serveraddr);

/* Send data from user input */
int BUFFERSIZE = 1024;
char buffer[BUFFERSIZE];
bzero(buffer, BUFFERSIZE);
memset(buffer, 0, BUFFERSIZE);
sprintf(buffer, "GET %s HTTP/1.1\n\nHost: %s\n\n\n", argv[2], argv[1]);
printf("HTTP Request Sent:\n%s", buffer);
int byte_sent = send(sockfd, buffer, strlen(buffer), 0);

/* Recieve Data */
printf("HTTP Response Recieved:\n");
bzero(buffer, BUFFERSIZE);
memset(buffer, 0, BUFFERSIZE);
int byte_recieved;
while((byte_recieved = recv(sockfd, buffer, BUFFERSIZE - 1, 0)) > 0) {
    buffer[byte_recieved] = '\0';
    printf("%s", buffer);
    memset(buffer, 0, BUFFERSIZE);
}

/* Close Socket */
close(sockfd);
printf("\nA socket is closed!\n");
return 0;
}

```

b: Examine your Program with Wireshark

Request:

http						
No.	Time	Source	Destination	Protocol	Length	Info
8	2.562041810	172.16.42.38	172.66.147.243	HTTP	113	GET /index.html HTTP/1.1
12	2.596995098	172.66.147.243	172.16.42.38	HTTP	71	HTTP/1.1 404 Not Found (text/html)

Frame 8: 113 bytes on wire (904 bits), 113 bytes captured (904 bits) on interface ens18, id 0

Ethernet II, Src: ProxmoxServe_43:9e:8f (bc:24:11:43:9e:8f), Dst: HewlettPacka_7a:b7:14 (88:e9:a4:7a:b7:14)

Internet Protocol Version 4, Src: 172.16.42.38, Dst: 172.66.147.243

Transmission Control Protocol, Src Port: 35770, Dst Port: 80, Seq: 1, Ack: 1, Len: 47

Hypertext Transfer Protocol

GET /index.html HTTP/1.1\r\n

Host: example.com\r\n

\r\n

[Full request URI: http://example.com/index.html]

[HTTP request 1/1]

[Response in frame: 12]

Difference with Lab 1:

The C client sent a minimal 113-byte request with only Host, whereas the web browser sent a 417-byte request packed with headers like User-Agent, Accept, and Accept-Encoding.

Response:

http						
No.	Time	Source	Destination	Protocol	Length	Info
8	2.562041810	172.16.42.38	172.66.147.243	HTTP	113	GET /index.html HTTP/1.1
12	2.596995098	172.66.147.243	172.16.42.38	HTTP	71	HTTP/1.1 404 Not Found (text/html)

Frame 12: 71 bytes on wire (568 bits), 71 bytes captured (568 bits) on interface ens18, id 0

Ethernet II, Src: HewlettPacka_7a:b7:14 (88:e9:a4:7a:b7:14), Dst: ProxmoxServe_43:9e:8f (bc:24:11:43:9e:8f)

Internet Protocol Version 4, Src: 172.66.147.243, Dst: 172.16.42.38

Transmission Control Protocol, Src Port: 80, Dst Port: 35770, Seq: 790, Ack: 48, Len: 5

Reassembled TCP Segments (794 bytes): #10(789), #12(5)]

Hypertext Transfer Protocol, has 2 chunks (including last chunk)

HTTP/1.1 404 Not Found\r\n

Date: Fri, 18 Sep 2026 16:33:01 GMT\r\n

Content-Type: text/html\r\n

Transfer-Encoding: chunked\r\n

Connection: keep-alive\r\n

Server: cloudflare\r\n

Age: 2345\r\n

cf-cache-status: HIT\r\n

CF-RAY: a3d1c001bad902c8-ORD\r\n

\r\n

[HTTP response 1/1]

[Time since request: 0.034953288 seconds]

[Request in frame: 8]

[Request URI: http://example.com/index.html]

HTTP chunked response

File Data: 559 bytes

Line-based text data: text/html (1 lines)

Difference with Lab 1:

The browser received compressed data, while the simple C client received uncompressed text and triggered a 404 Not Found response due to missing browser headers.

c: Security Analysis

i: Input data of program

1. Command-line argument 1 (argv[1]): User given server name passed into main()
2. Command-line argument 2 (argv[2]): User given resource path passed into main()
3. serveraddr: Server IP and connection metadata returned from the network by getaddrinfo().
4. buffer: External web page data received over the network socket via recv().

- ii: Review whether data can pose a security risk

```
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$ ./myhttpclient "%x %s" /
HTTP Client Program by Grant Harvey
Server Name = %x %s, Path = /
A socket is opened!
getaddrinfo error: Name or service not known
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/Lab2$
```

Format String Attack:

- Test: Passing format specifiers ("%x %s") as argv[1].
- Result: The output explicitly prints as intended
- Format string attacks are prevented by hardcoding %s specifiers in all printf() and sprintf() calls so user inputs are treated as literal text.

[illegible]

Buffer Overflow Attack:

- Test: Passing an excessively long string to overflow internal buffers.
- Result: Caught too long strings in code safely
- Buffer overflows are avoided by validating command-line argument lengths with `strlen()` and capping network reads to `BUFFERSIZE - 1` bytes